Offshore Drilling RAG

Case study: a citation-first question-answering system over offshore drilling and maritime manuals

Role	Approach	Tests	Deployment
Sole developer	Test-driven, 6 phases	115 unit + integration suite	Local (Ollama) + HF Space demo

1. The problem

Offshore drilling engineers work from long technical manuals: drilling procedures, well-control equipment, offshore safety and classification guidance. Finding one fact (a BOP pressure rating, a mud-weight rule, a safety clause) means paging through hundreds of PDF pages. A general chatbot is faster but unusable here: in a safety-critical domain a confident, invented answer is worse than no answer.

The brief therefore had three hard requirements:

- **No hallucination.** Answers may come only from retrieved manual text. If the manuals do not cover the question, the system must refuse.
- **Verifiable.** Every claim carries a citation to file and page, e.g. `[Drilling_Manual.pdf p.12]`, with the source excerpt shown so an engineer can check it.
- **Local and private.** Embeddings, index and LLM all run on the engineer's own machine; no document text leaves it.

A further retrieval problem shaped the design: manual questions mix *meaning* ("what seals the well during a kick?") with *exact tokens* ("API 16A", "5000 psi", "BOP"). No single search method handles both well.

2. Tech stack

Layer	Technology	Why
Parsing	pypdf, pdfplumber	Page-level text keeps page numbers for citations; tables are converted to Markdown so rows stay readable to the model.
Chunking	tiktoken (~500 tokens, 50 overlap)	Fits the 512-token limit of the embedding model; overlap stops facts being cut at a boundary. Each chunk keeps source_file, page_number, chunk_id.
Dense vectors	BAAI/bge-small-en-v1.5 + ChromaDB (cosine)	Semantic match: finds the right section even when the wording differs from the question.
Keyword search	BM25 (rank_bm25)	Lexical match: reliable on acronyms, standards, part numbers and figures that embeddings blur.
Fusion	Reciprocal Rank Fusion (k = 60)	Merges both rankings by position only, so incompatible score scales never need normalising.
Generation	Ollama, llama3.1:8b, temperature 0	Runs locally; strict system prompt allows only the labelled context blocks.
Serving	FastAPI /query, Streamlit UI	JSON {answer, sources[]}; UI shows one expander per cited source.
Quality	pytest (TDD), Python 3.10+ type hints	Fake embedder and stub LLM keep unit tests fast and offline.

3. How retrieval works

Every question runs through **both** retrievers, taking the top 20 chunks from each. The two ranked lists are merged with Reciprocal Rank Fusion:

```
score(chunk) = Σ over lists containing it of 1 / (60 + rank)
```

A chunk ranked highly by both semantic and keyword search gets the largest fused score; the top 5 go to the model. Because RRF ignores raw scores, there is no fragile normalisation between BM25 values and cosine similarities.

4. Guarding against hallucination

Hallucination is handled in code, not left to the prompt alone. Three independent gates sit between a question and an answer:

- **Relevance gate.** The fused RRF score only orders results, so each chunk also keeps its raw cosine `dense_score`. If no chunk reaches 0.6, the API refuses *without calling the LLM*. The threshold came from measured values: about 0.49 for off-topic questions, 0.73 to 0.86 for on-topic ones.
- **Strict prompt.** Each context block is labelled with its exact citation string, and the model is told to answer only from those blocks or say it cannot.
- **Citation check.** After generation, the API verifies that every `[file p.N]` in the answer matches a chunk that was actually retrieved. Any invented citation turns the answer into a refusal.

5. Results and lessons

- Five of six manuals indexed and searchable end to end; answers return with page-level citations and source excerpts.
- Off-topic questions are refused before the LLM runs, saving latency and removing the main hallucination path.
- Public demo deployed as a Docker Hugging Face Space (Inference API backend, per-session rate limit, token kept as a Space secret); the main code stays fully local on Ollama.
- **Known limit:** one manual is a scanned image with no text layer. It is deliberately excluded rather than OCR'd, because OCR errors would undermine citation accuracy.
- **Lesson:** hybrid search was the single biggest retrieval win. Dense search alone missed exact standards and acronyms; BM25 alone missed paraphrased questions. RRF combined them with three lines of arithmetic.
- **Lesson:** dependency injection (embedder, retrievers, LLM) made test-first development practical: 115 fast tests run offline in about 15 seconds.

6. Code highlight: hybrid retrieval with RRF

From `src/rag/hybrid.py` (comments expanded for this case study). Retrievers are injected via a `Protocol`, so tests pass in-memory fakes and the fusion logic is tested without any model or database.

```
        from dataclasses import replace

        from rag.models import RetrievedChunk


        def reciprocal_rank_fusion(
            ranked_lists: list[list[RetrievedChunk]], k: int = 60, top_n: int = 5
        ) -> list[RetrievedChunk]:
            """Merge several ranked result lists into one using Reciprocal Rank Fusion.

            RRF looks only at each chunk's *position* in a list, never at the raw
            score. That matters here because BM25 scores (unbounded word counts) and
            cosine similarities (-1..1) live on different scales and cannot be
            compared directly.
            """
            scores: dict[str, float] = {}
            first_seen: dict[str, RetrievedChunk] = {}

            for ranked in ranked_lists:
                # rank starts at 1: the best hit in each list contributes 1 / (k + 1).
                for rank, chunk in enumerate(ranked, start=1):
                    # A chunk found by BOTH retrievers collects two contributions,
                    # so agreement between keyword and semantic search wins.
                    scores[chunk.chunk_id] = scores.get(chunk.chunk_id, 0.0) + 1 / (k + rank)
                    # Keep the first copy so its text and citation metadata survive.
                    first_seen.setdefault(chunk.chunk_id, chunk)

            # Highest fused score first; chunk_id breaks ties so output is deterministic.
            order = sorted(scores, key=lambda cid: (-scores[cid], cid))

            # Return immutable copies whose `score` field now holds the fused RRF score.
            return [replace(first_seen[cid], score=scores[cid]) for cid in order[:top_n]]


        class HybridRetriever:
            """Runs dense and BM25 search side by side, then fuses the rankings."""

            def __init__(self, dense: Retriever, bm25: Retriever) -> None:
                self._dense = dense   # ChromaDB + bge-small-en-v1.5 embeddings
                self._bm25 = bm25     # rank_bm25 keyword index

            def retrieve(
                self, query: str, k_each: int = 20, top_n: int = 5
            ) -> list[RetrievedChunk]:
                dense_results = self._dense.search(query, k_each)
                ranked_lists = [dense_results, self._bm25.search(query, k_each)]

                # RRF scores only order results; they say nothing about relevance.
                # Keep each chunk's raw cosine similarity so the API can refuse
                # off-topic questions (no chunk >= 0.6) before calling the LLM.
                dense_scores = {c.chunk_id: c.score for c in dense_results}
                fused = reciprocal_rank_fusion(ranked_lists, top_n=top_n)
                return [replace(c, dense_score=dense_scores.get(c.chunk_id)) for c in fused]
```

Repository: github.com/Riz695/offshore-drilling-rag · Demo: huggingface.co/spaces/Riz695/offshore-drilling-rag